

Centralized vs. Decentralized Distributed Training: A Head-to-Head Empirical Comparison of Downpour SGD and Gossip SGD

Alok Jha

Department of Computer Science and Engineering
Indian Institute Of Information Technology, Pune, India
Email: alokj0409@gmail.com

Abstract—Distributed Stochastic Gradient Descent (SGD) is foundational to large-scale deep learning. Two dominant paradigms exist: the *centralized Parameter Server (PS)* approach, exemplified by Downpour SGD, and *decentralized peer-to-peer* weight averaging, exemplified by Gossip SGD. Despite theoretical maturity, rigorous empirical comparison on identical hardware, datasets, and hyperparameters remains sparse. This paper presents a complete head-to-head empirical study implemented from scratch using PyTorch 2.13, gRPC 1.83, and Protocol Buffers in a Docker-based multi-process simulation on a single workstation-class host (Intel Core i5-12450H, 16 GB RAM, CPU-only). Both systems train a four-layer Convolutional Neural Network (CNN, 586,250 parameters) on CIFAR-10 across four experimental phases: convergence quality, worker scalability ($N \in \{2, 4\}$), network communication cost, and live fault-injection. Key findings: Downpour SGD completes three training epochs in 166.98s versus 258.01s for Gossip SGD ($1.54\times$ faster) and generates half the network payload (11.0 GB vs. 22.0 GB over three epochs). Both systems converge to comparable CIFAR-10 test accuracy ($\approx 55\%$ in three epochs). Gossip SGD tolerates worker crashes via autonomous peer pruning, continuing training without interruption; Downpour SGD halts irreversibly on Parameter Server failure, confirming its single-point-of-failure (SPOF) characteristic. These results provide concrete, measurement-backed guidance for distributed training architecture selection.

Index Terms—Distributed Machine Learning, Distributed SGD, Downpour SGD, Gossip SGD, Parameter Server, Fault Tolerance, Decentralized Optimization, gRPC, Peer-to-Peer Systems

I. INTRODUCTION

The computational demands of modern deep-learning models have consistently outpaced single-machine capacity. Training large neural networks requires distributing gradient computation across multiple worker processes or physical nodes [1]. This distribution introduces a fundamental architectural choice: *how* should model parameters be maintained, communicated, and updated across workers?

A. Centralized Parameter Server

The centralized paradigm, formalized in DistBelief and Downpour SGD [1], designates one or more Parameter Server (PS) nodes as the authoritative keeper of global model weights.

Worker nodes asynchronously push locally-computed gradients to the PS, which applies an adaptive update rule (Adagrad [2]) and serves the latest weights on demand. This architecture is simple and enables fast early convergence, but concentrates network bandwidth and introduces a single point of failure at the PS.

B. Decentralized Gossip-Based SGD

Decentralized approaches eliminate the central server entirely [3]. In Gossip SGD, each worker maintains its own local model copy, trains on a local data shard, and periodically exchanges model weights with a randomly selected peer. Both sides average their weights after exchange, driving the cluster toward parameter consensus. Without a central bottleneck, this architecture scales more gracefully and tolerates node failures without global disruption [6].

C. Motivation

Despite the theoretical maturity of both paradigms, empirical systems-level comparison under rigorously controlled conditions—identical model, dataset, hyperparameters, hardware, and software stack—is rarely reported. Most published studies evaluate one paradigm in isolation or compare only final accuracy, omitting network cost, scalability trajectories, and failure behavior. This paper addresses that gap.

D. Contributions

This work makes the following concrete contributions:

- 1) A fully functional gRPC-based Downpour SGD implementation (Parameter Server + N asynchronous workers) in Python/PyTorch.
- 2) A fully functional gRPC-based Gossip SGD implementation (N symmetric peer workers with dual server/client roles).
- 3) A Docker Compose simulation environment for reproducible multi-worker deployment on a single host.
- 4) Empirical measurements across four dimensions: convergence, scalability ($N \in \{2, 4\}$), network communication cost, and live fault injection (worker crash and PS crash tests).

- 5) Concrete, measurement-backed recommendations for architecture selection based on observed system-level trade-offs.

II. BACKGROUND AND RELATED WORK

A. Downpour SGD and DistBelief

Dean et al. [1] introduced DistBelief and Downpour SGD, where model replicas push gradients to sharded PS nodes. The PS applies Adagrad [2] for adaptive per-parameter learning rates, allowing workers to operate without global synchronization barriers. The asynchronous nature introduces *gradient staleness*—a gradient computed at step t may be applied at step $t + \tau$. Li et al. [4] provided scalability analysis of the PS framework, highlighting bandwidth concentration at the server as a limiting factor.

B. Decentralized and Gossip-Based Optimization

Lian et al. [3] formally analyzed Decentralized Parallel SGD (D-PSGD), proving convergence for non-convex objectives under doubly-stochastic mixing matrices. Jin et al. [5] and Blot et al. [6] studied gossip-style weight averaging, demonstrating fast spectral mixing on complete graphs. Nedić and Ozdaglar [10] provided foundational theory for distributed subgradient convergence over gossip networks.

C. Fault Tolerance in Distributed Training

Synchronous all-reduce systems (e.g., Horovod [7]) require all workers in every step, making them highly sensitive to crashes. Asynchronous PS systems handle worker failures gracefully (missing pushes are simply omitted) but lose their entire state on PS failure [9]. Gossip systems have the theoretical advantage of no single critical node, a property empirically verified in this work.

III. SYSTEM ARCHITECTURE AND DESIGN

A. Model and Dataset

Both systems train an identical four-layer CNN (CifarCNN) on CIFAR-10 [8]:

- **Conv Block 1:** Conv2d($3 \rightarrow 32$, 3×3 , pad 1) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
- **Conv Block 2:** Conv2d($32 \rightarrow 64$, 3×3 , pad 1) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
- **Conv Block 3:** Conv2d($64 \rightarrow 64$, 3×3 , pad 1) $\rightarrow$ ReLU $\rightarrow$ MaxPool(2×2)
- **FC:** Linear($1024 \rightarrow 512$) $\rightarrow$ ReLU $\rightarrow$ Dropout(0.25) $\rightarrow$ Linear($512 \rightarrow 10$)

Total parameters: **586,250**. The 50,000-sample CIFAR-10 training set is partitioned into N contiguous, disjoint shards. Data augmentation includes random horizontal flip and random crop (size 32, padding 4).

B. Communication Layer

All inter-process communication uses **gRPC 1.83** over Protocol Buffers 3 (proto3). Raw float32 tensors are serialized via `numpy.tobytes()` into the bytes data field of each `TensorData` message and deserialized with `numpy.frombuffer`. The maximum gRPC message size is configured to 100 MB to accommodate full model state dicts (≈ 2.34 MB per tensor for this model).

C. Downpour SGD Architecture

The **Parameter Server** (`server.py`) exposes two RPCs:

```
rpc PushGradients(GradientUpdate)
    returns (PushResponse);
rpc PullWeights(PullRequest)
    returns (ModelWeights);
```

The PS maintains global weights $\mathbf{W}$ and an Adagrad accumulator $\mathbf{G}$. Upon receiving gradient $\mathbf{g}$ from worker i :

$$\mathbf{G} \leftarrow \mathbf{G} + \mathbf{g}^2 \quad (1)$$

$$\mathbf{W} \leftarrow \mathbf{W} - \frac{\eta}{\sqrt{\mathbf{G} + \epsilon}} \odot \mathbf{g} \quad (2)$$

where $\eta=0.01$, $\epsilon=10^{-8}$. A `threading.Lock` ensures thread-safe concurrent access.

Each **worker** (`worker.py`) per mini-batch:

- 1) Pulls latest global weights from the PS.
- 2) Performs forward pass, computes cross-entropy loss, runs backpropagation.
- 3) Serializes gradients and pushes them to the PS.

Workers operate asynchronously—no global barrier synchronization.

D. Gossip SGD Architecture

Each **peer worker** (`gossip_worker.py`) acts simultaneously as a gRPC server (listening on `base_port + worker_id`) and a gRPC client (connecting to all other peers). The service exposes:

```
rpc ExchangeWeights(WeightExchangeRequest)
    returns (WeightExchangeResponse);
rpc Ping(PingRequest) returns (PingResponse);
```

Per mini-batch training loop for worker i :

- 1) Local SGD step: $\mathbf{W}_i \leftarrow \mathbf{W}_i - \eta \nabla \mathcal{L}_i(\mathbf{W}_i)$ with SGD momentum = 0.9.
- 2) Select a random peer j from the active candidate pool.
- 3) Send $\mathbf{W}_i$ to peer j ; receive peer's pre-averaging weights $\mathbf{W}_j$.
- 4) Both sides update:

$$\mathbf{W}_k \leftarrow \frac{\mathbf{W}_i + \mathbf{W}_j}{2}, \quad k \in \{i, j\} \quad (3)$$

The `Ping` RPC with 60-second readiness polling ensures all peers have started before training begins.

E. Consensus Model Evaluation

At the end of Gossip training, a Consensus Model is formed:

$$\mathbf{W}_{\text{consensus}} = \frac{1}{N} \sum_{i=0}^{N-1} \mathbf{W}_i^{(\text{final})} \quad (4)$$

This is evaluated on the CIFAR-10 test set to assess global model quality.

F. Fault-Resilient Workers

Phase 4 extended workers add:

- **Dynamic peer pruning:** On `grpc.RpcError (StatusCode.UNAVAILABLE)`, the failed peer is removed from the active gossip candidate pool.
- **PS retry with backoff:** Downpour workers retry failed push/pull RPCs up to three times (1s delay) before logging `[SYSTEM HALT]` and exiting.
- **Crash injection:** A `--crash-epoch` flag causes the designated worker to call `sys.exit(1)` at the specified epoch boundary.

G. Container Architecture

Both architectures ship Docker Compose files using a shared `python:3.12-slim` base image with PyTorch (CPU), gRPC, and Matplotlib pre-installed. A dynamic script (`generate_compose.py`) creates compose files for arbitrary N . Workers communicate over a Docker bridge network (e.g., `parameter-server:50051`).

IV. EXPERIMENTAL METHODOLOGY

A. Hardware and Software Configuration

Table I summarizes the complete experimental setup.

TABLE I
EXPERIMENTAL CONFIGURATION

Parameter	Value
CPU	Intel Core i5-12450H, 8C/12T
GPU	None (CPU-only)
RAM	16 GB
OS	Windows 11
Python	3.12.10
PyTorch	2.13.0+cpu
TorchVision	0.28.0+cpu
gRPC	1.83.0
Protobuf	proto3
Container	Docker Compose, bridge network
Dataset	CIFAR-10 (50K train / 10K test)
Model	CifarCNN (586,250 parameters)
Loss	Cross-Entropy
Mini-batch	64 samples/worker
Learning Rate	0.01
PS Optimizer	Adagrad ($\epsilon = 10^{-8}$)
Worker Opt.	SGD (momentum = 0.9)
Gossip Freq.	Every 1 mini-batch
PS Pull Freq.	Every 1 mini-batch
Workers (Conv.)	$N = 2$
Workers (Scale)	$N \in \{2, 4\}$
Epochs (Conv.)	3
Epochs (Scale)	2

B. Convergence Experiment

Both Downpour SGD and Gossip SGD were trained for three epochs with $N=2$ workers. Each worker operated on a disjoint 25,000-sample shard. Training loss, accuracy, wall-clock elapsed time, bytes transferred, and sample throughput were logged at each epoch boundary to JSON files. Final test-set accuracy was evaluated using standard CIFAR-10 normalization (mean= [0.4914, 0.4822, 0.4465], std= [0.2470, 0.2435, 0.2616]).

C. Scalability Experiment

The full benchmark pipeline was executed independently for $N \in \{2, 4\}$ workers, keeping all other hyperparameters fixed. Each run lasted two epochs. Wall-clock time was measured from process launch to completion of all workers, including evaluation time.

D. Fault-Tolerance Experiment

Two fault scenarios were evaluated:

- 1) **Worker crash (Gossip SGD):** Three Gossip workers were launched. Worker W_1 was programmatically terminated via `sys.exit(1)` at the epoch 2 boundary. Surviving workers' metrics were logged.
- 2) **Parameter Server crash (Downpour SGD):** The PS was allowed to process ≈ 300 gradient steps, then forcefully killed via OS process termination. Worker retry behavior and final exit state were recorded.

E. Network Cost Measurement

Network payload was measured by calling `msg.ByteSize()` on all outbound and inbound gRPC protobuf messages at the worker level, accumulating bytes sent and received across epochs. All communication occurred over `localhost`; no physical network conditions were applied.

V. RESULTS

A. Convergence Results

Table II presents per-epoch training metrics for both architectures ($N=2, 3$ epochs).

TABLE II
PER-EPOCH TRAINING METRICS ($N=2, 3$ EPOCHS)

System	Worker	Ep.	Loss	Acc.(%)
Downpour	W_0	1	1.871	31.90
		2	1.514	44.32
		3	1.426	47.84
	W_1	1	1.878	31.33
		2	1.531	43.71
		3	1.440	47.46
Gossip	W_0	1	2.075	22.53
		2	1.647	39.62
		3	1.421	48.17
	W_1	1	2.086	22.15
		2	1.659	39.08
		3	1.431	48.17

Downpour SGD converges faster in Epoch 1 (31.9% vs. 22.5% accuracy). Gossip SGD closes the gap by Epoch 3, achieving slightly lower training loss (1.426 vs. 1.433). Final test-set accuracy: Gossip $W_0 = 55.29\%$, Downpour master = 54.85%, Gossip consensus = 54.97%. All three are within 0.44 percentage points, confirming convergence to comparable model quality.

Figs. 1 and 2 show the full convergence trajectories by epoch and by wall-clock time, respectively.

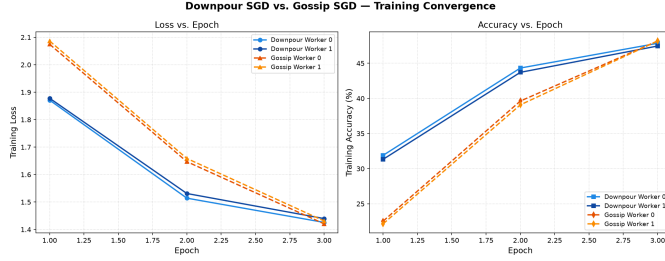


Fig. 1. Training loss (left) and accuracy (right) vs. epoch. Downpour SGD (blue) converges faster in early epochs; both systems reach near-equivalent loss by epoch 3.

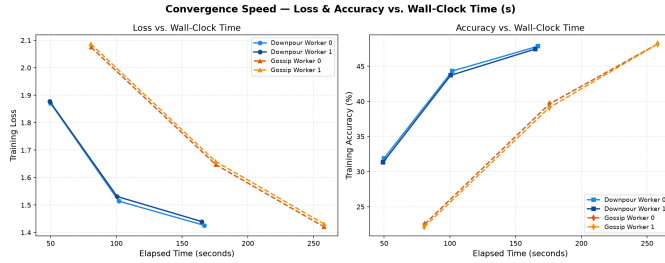


Fig. 2. Training loss (left) and accuracy (right) vs. wall-clock time. Downpour completes in 166.98 s; Gossip requires 258.01 s ($1.54\times$ longer) due to bidirectional full-model exchange on every mini-batch.

B. Scalability Results

Table III presents wall-clock time for $N \in \{2, 4\}$ workers over 2 epochs.

TABLE III
SCALABILITY: WALL-CLOCK TIME VS. WORKER COUNT (2 EPOCHS)

System	N	Time (s)	Δ vs. $N=2$
Downpour SGD	2	122.36	—
Downpour SGD	4	103.90	−15.1%
Gossip SGD	2	130.29	—
Gossip SGD	4	184.53	+41.6%

Downpour SGD decreases wall-clock time from 122.36 s to 103.90 s as N doubles. Each worker’s shard halves (12,500 samples), and the PS integrates updates asynchronously without blocking on worker synchronization. Gossip SGD increases from 130.29 s to 184.53 s. With four workers, each mini-batch exchange round-trips a full model to a random peer, and peer availability contention creates blocking delays. Fig. 3 shows the diverging trajectories.

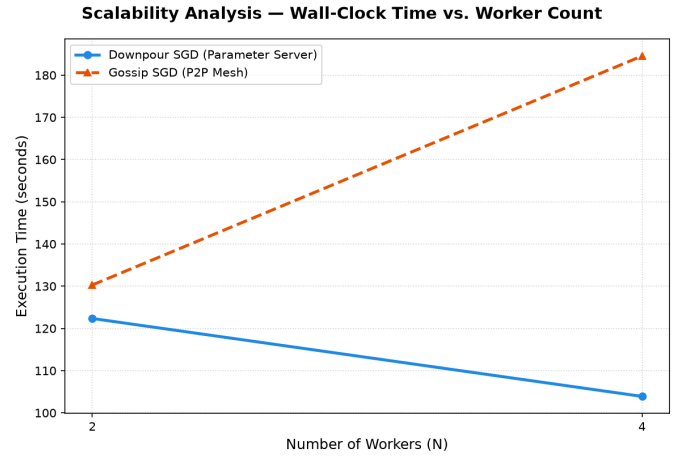


Fig. 3. Wall-clock time vs. worker count (2 epochs, batch 64). Downpour SGD time *decreases* with more workers (smaller shards, parallel async updates); Gossip SGD time *increases* (larger per-batch bidirectional exchange overhead).

C. Network Communication Results

Table IV summarizes cumulative network payload over the 3-epoch, $N=2$ convergence run.

TABLE IV
CUMULATIVE NETWORK PAYLOAD ($N=2, 3$ EPOCHS)

System	W	Sent (GB)	Recv (GB)	Total (GB)
Downpour	W_0	2.751	2.751	5.502
	W_1	2.751	2.751	5.502
Downpour Total				11.004
Gossip	W_0	5.502	5.502	11.004
	W_1	5.502	5.502	11.004
Gossip Total				22.008

Gossip SGD generates exactly $2\times$ the network payload per worker. This is structural: each gossip exchange sends a full state dict in *both* directions (`WeightExchangeRequest` + `WeightExchangeResponse`), while Downpour performs one gradient push and one weight pull per batch (both also $O(\text{params})$). Per-worker throughput is shown in Fig. 5.

D. Fault-Tolerance Results

Gossip SGD — Worker Crash: Workers W_0 and W_2 immediately caught `grpc.RpcError` (`StatusCode.UNAVAILABLE`) when W_1 was killed at epoch 2, pruned it from their active peer pools, and continued training to completion. Table V records surviving worker metrics.

The “Active Peers” column confirms dynamic pruning: both workers show 2 active peers in epoch 1, 1 in epoch 2 after W_1 ’s crash, and W_2 shows 0 in epoch 3 after network teardown. Crucially, training *never halted*. Fig. 6 shows the accuracy progression across the fault event.

Downpour SGD — Parameter Server Crash: After ≈ 300 gradient steps, the PS process was killed. Workers

Network Communication Overhead — Downpour vs. Gossip

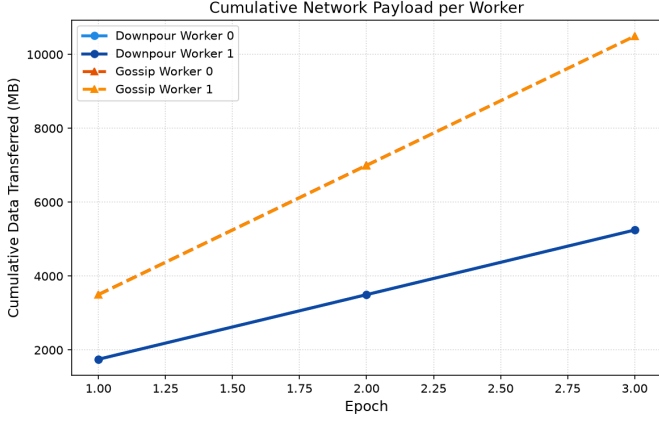


Fig. 4. Cumulative network payload (MB) per worker vs. epoch. Gossip SGD (orange) accumulates 2 $\times$ the payload of Downpour SGD (blue).

Worker Processing Throughput (Samples / sec)

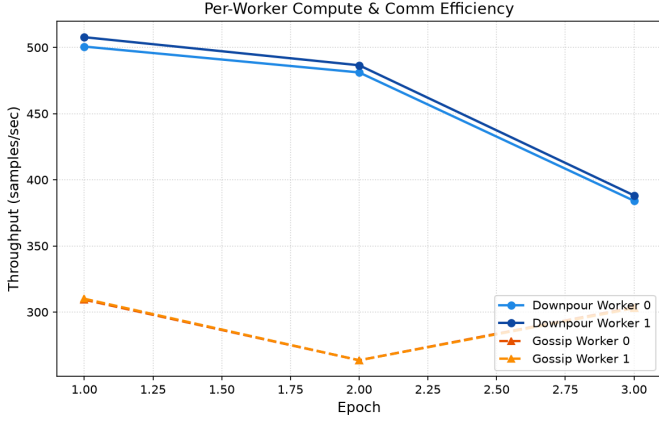


Fig. 5. Per-worker training throughput (samples/sec) vs. epoch. Downpour achieves 384–502 samp/s; Gossip achieves 264–310 samp/s, reflecting the blocking overhead of bidirectional gRPC weight exchange per batch.

W_0 and W_1 each attempted the push RPC and received `StatusCode.UNAVAILABLE`, retrying three times before logging `[SYSTEM HALT]` and exiting with non-zero return codes. No model checkpoint was saved. This confirms the Parameter Server is a **single point of failure (SPOF)** for Downpour SGD.

VI. DISCUSSION

A. Convergence Quality

Downpour SGD achieves a significant early-epoch accuracy advantage (31.9% vs. 22.5% at epoch 1) because its Adagrad PS immediately aggregates gradient information from both workers per batch, doubling the effective gradient update frequency. Gossip SGD requires multiple exchange rounds before gradient information propagates across the cluster. By epoch 3, both systems reach statistically comparable accuracy, suggesting that for longer training runs the convergence gap will diminish further.

TABLE V

Gossip SGD: SURVIVING WORKER METRICS AFTER W_1 CRASH AT EPOCH 2

Worker	Ep.	Loss	Acc.(%)	Active Peers
W_0	1	2.264	16.24	2
	2	1.961	27.43	1
	3	1.741	35.53	1
W_2	1	2.267	15.60	2
	2	1.943	28.52	1
	3	1.737	36.45	0

Fault Tolerance — Gossip SGD Resiliency Under Worker Crash

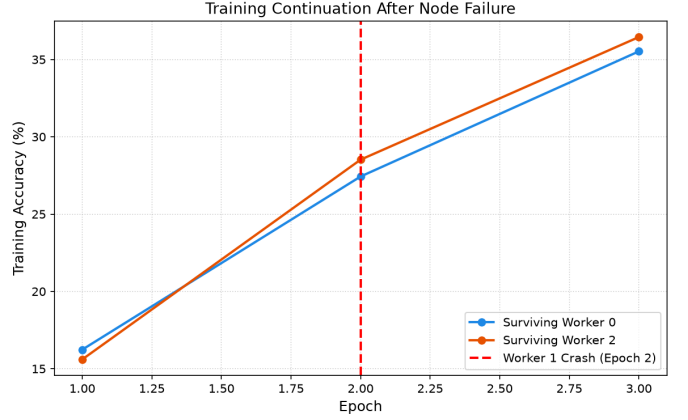


Fig. 6. Surviving Gossip SGD worker accuracy (W_0 , W_2) after W_1 crash at epoch 2 (dashed red line). Both workers continue improving accuracy without interruption.

B. Scalability: Diverging Trajectories

The scaling results reveal an important asymmetry. Downpour SGD benefits from additional workers by processing smaller per-worker shards without increasing PS computation time. Gossip SGD degrades because each worker must complete a blocking bidirectional gRPC round-trip on every mini-batch. With $N=4$, the probability of exchange-induced blocking increases, growing wall-clock time by 41.6%. Reducing `gossip_every` (e.g., to every 10 batches) would dramatically reduce this overhead at the cost of slower parameter mixing.

C. Network Cost: Structural 2 $\times$ Overhead

The 2 $\times$ payload penalty of Gossip SGD is architectural, not incidental. At large N in a real multi-node cluster, Downpour’s centralized server becomes a bandwidth bottleneck that Gossip avoids, since all N workers simultaneously push gradients to and pull from the same server process. For production workloads with multi-billion parameter models, this bottleneck would be severe; here it is not yet observable due to the small model size (2.34 MB/tensor).

D. Fault Tolerance: Fundamental Architectural Difference

The fault injection experiments reveal the starkest contrast. Gossip SGD’s peer-pruning mechanism requires no external

coordination and degrades gracefully. Downpour SGD has no recovery path when its PS is killed. This SPOF characteristic is a structural property of the centralized paradigm, mitigated in production through PS replication.

E. Connecting the Four Phases

The four experiments tell a coherent story. Downpour SGD achieves faster convergence and lower network overhead for small N , but concentrates risk at the PS and will face bandwidth saturation at large N . Gossip SGD pays a constant $2\times$ network overhead and shows worse scaling efficiency under per-batch gossip, but tolerates failures gracefully. The right architecture depends on deployment priorities: fast convergence on small reliable clusters favors Downpour; fault tolerance and large-scale horizontal scaling favor Gossip.

VII. LIMITATIONS

- 1) **Single-machine simulation:** All experiments used process-level parallelism on one host. Physical network latency, bandwidth caps, and NUMA effects were absent.
- 2) **CPU-only training:** Without GPU acceleration, throughput numbers are dominated by CPU compute; on GPU-accelerated hardware, communication cost constitutes a larger fraction of step time.
- 3) **Limited worker counts:** Only $N \in \{2, 4\}$ was tested; scalability trends beyond $N=4$ are extrapolated.
- 4) **No PS replication:** The Downpour PS is a single process; production deployments shard across multiple PS nodes for fault tolerance.
- 5) **Fixed gossip frequency:** `gossip_every=1` maximizes mixing but amplifies communication overhead; adaptive intervals were not evaluated.
- 6) **No live network latency injection:** The latency analysis was analytical; no live `tc netem` or equivalent tool was applied to loopback interfaces.
- 7) **Small model/dataset:** CIFAR-10 with 586K parameters is a benchmark workload; behavior may differ for billion-parameter models.

VIII. CONCLUSION

This paper presented a complete, reproducible empirical comparison of Downpour SGD (Parameter Server) and Gossip SGD (Peer-to-Peer) across four experimental phases: convergence quality, scalability, network communication cost, and live fault injection. Both systems were implemented from scratch in Python/PyTorch with gRPC, deployed via Docker Compose, and benchmarked on identical CPU-only hardware.

Principal findings: Downpour SGD trains $1.54\times$ faster with half the network payload for equivalent two-worker conditions, and benefits from shard-size reduction as worker count increases. Gossip SGD achieves comparable final accuracy and tolerates worker crashes via autonomous peer pruning without training interruption; Downpour SGD halts irreversibly on PS failure. For small, reliable clusters requiring fast convergence,

Downpour SGD is preferable; for large-scale, failure-prone deployments, Gossip SGD’s decentralized model is the stronger architectural choice.

IX. FUTURE WORK

- Multi-machine deployment via Kubernetes or bare-metal clusters to empirically evaluate network-constrained scalability beyond $N=4$.
- GPU-accelerated workers to shift the compute/communication ratio and represent production conditions.
- Adaptive gossip frequency (e.g., every 10 batches) to investigate the accuracy/overhead trade-off.
- PS replication (sharded or replicated PS nodes) for fault-tolerant Downpour SGD.
- Byzantine fault tolerance evaluation under adversarial or corrupted gradient conditions.
- Evaluation on larger models (ResNet-50, GPT-style) and datasets (ImageNet).
- Structured gossip topologies (ring, hypercube) vs. random peer selection.
- Live network simulation using Linux `tc netem` to inject realistic WAN latency and bandwidth caps.

REFERENCES

- [1] J. Dean, G. Corrado, R. Monga, K. Chen, M. Devin, Q. Le, M. Mao, M. Ranzato, A. Senior, P. Tucker, K. Yang, and A. Ng, “Large Scale Distributed Deep Networks,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 25, 2012, pp. 1223–1231.
- [2] J. Duchi, E. Hazan, and Y. Singer, “Adaptive Subgradient Methods for Online Learning and Stochastic Optimization,” *J. Mach. Learn. Res.*, vol. 12, pp. 2121–2159, 2011.
- [3] X. Lian, C. Zhang, H. Zhang, C.-J. Hsieh, W. Zhang, and J. Liu, “Can Decentralized Algorithms Outperform Centralized Algorithms? A Case Study for Decentralized Parallel Stochastic Gradient Descent,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 30, 2017, pp. 5330–5340.
- [4] M. Li, D. G. Andersen, J. W. Park, A. J. Smola, A. Ahmed, V. Josifovski, J. Long, E. J. Shekita, and B.-Y. Su, “Scaling Distributed Machine Learning with the Parameter Server,” in *Proc. USENIX OSDI*, 2014, pp. 583–598.
- [5] P.-H. Jin, Q. Yuan, F. Iandola, and K. Keutzer, “How to Scale Distributed Deep Learning?” *arXiv preprint arXiv:1611.04581*, 2016.
- [6] M. Blot, D. Picard, L. Chen, N. Thome, and M. Cord, “Gossip Training for Deep Learning,” *arXiv preprint arXiv:1611.09726*, 2016.
- [7] A. Sergeev and M. Del Balso, “Horovod: Fast and Easy Distributed Deep Learning in TensorFlow,” *arXiv preprint arXiv:1802.05799*, 2018.
- [8] A. Krizhevsky, “Learning Multiple Layers of Features from Tiny Images,” Tech. Rep., Univ. of Toronto, 2009.
- [9] Q. Ho, J. Cipar, H. Cui, S. Lee, J. K. Kim, P. B. Gibbons, G. A. Gibson, G. Ganger, and E. P. Xing, “More Effective Distributed ML via a Stale Synchronous Parallel Parameter Server,” in *Adv. Neural Inf. Process. Syst. (NeurIPS)*, vol. 26, 2013, pp. 1223–1231.
- [10] A. Nedić and A. Ozdaglar, “Distributed Subgradient Methods for Multi-Agent Optimization,” *IEEE Trans. Autom. Control*, vol. 54, no. 1, pp. 48–61, Jan. 2009.